

IPC?

In a multitasking operating system like Linux, processes often need to exchange data or coordinate actions. Interprocess Communication (IPC) provides mechanisms for processes to:

- Share information
- Distribute work (e.g., producer–consumer)
- Synchronize execution

IPC is fundamental for building complex, modular systems where independent processes cooperate.

2. Overview of IPC Mechanisms

Linux supports several IPC methods. Each has strengths and trade-offs:

Mechanism	Description
Pipe	Unidirectional byte stream; used between related processes.
FIFO (named pipe)	Similar to pipe but identified by a filesystem path; unrelated processes can communicate.
Shared Memory	Fastest IPC; multiple processes map the same memory region. Needs synchronization.
Message Queues	Processes exchange messages (with type, priority).
Signals	Asynchronous notifications; limited data transfer.
Sockets	Communication over network or locally (Unix domain sockets).

This guide focuses on **pipes** and **shared memory** – two widely used and contrasting approaches.

3. Pipes

3.1 Unnamed Pipes

An unnamed pipe is a unidirectional data channel created with the `pipe()` system call. It is typically used for communication between a parent and its child process (after `fork()`).

Key System Calls

- `int pipe(int pipefd[2])`
Creates a pipe. `pipefd[0]` is the read end, `pipefd[1]` is the write end. Returns 0 on success, -1 on error.
- `read(fd, buffer, count)` and `write(fd, buffer, count)` for data transfer.
- `close(fd)` to close an end.

How It Works

1. Parent calls `pipe()` to create a pipe.
2. Parent calls `fork()` to create a child.
Both processes now have copies of the file descriptors.
3. One end is closed in each process to enforce unidirectional flow.
Example: parent closes read end (`fd[0]`), child closes write end (`fd[1]`).
Data flows from parent → child.
4. When done, both processes close the remaining ends.

Example: Parent Writes, Child Reads

```
c
// pipe_demo.c
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <string.h>
#include <stdlib.h>

int main() {
    int fd[2];
    pid_t pid;
    char write_msg[] = "Hello from parent!";
    char read_msg[100];

    if (pipe(fd) == -1) {
        perror("pipe");
        exit(EXIT_FAILURE);
    }

    pid = fork();
    if (pid < 0) {
        perror("fork");
        exit(EXIT_FAILURE);
    }

    if (pid > 0) { // Parent process
        close(fd[0]); // Close unused read end
        write(fd[1], write_msg, strlen(write_msg) + 1);
        close(fd[1]); // Close write end after writing
    }
}
```

```

        wait(NULL);                // Wait for child to finish
    } else { // Child process
        close(fd[1]);               // Close unused write end
        read(fd[0], read_msg, sizeof(read_msg));
        printf("Child received: %s\n", read_msg);
        close(fd[0]);              // Close read end
    }

    return 0;
}

```

Compile & run:

```

bash
gcc pipe_demo.c -o pipe_demo
./pipe_demo

```

Output:

```

text
Child received: Hello from parent!

```

Limitations

- Half-duplex (data flows only one way). If two-way communication is needed, two pipes are required.
- Can only be used between processes that share a common ancestor (created via `fork`).
- Data is a byte stream; no message boundaries.

3.2 Named Pipes

A named pipe (FIFO) exists as a special file in the filesystem. Unrelated processes can open it and communicate. It behaves like a pipe but has a name.

Key System Calls

- `int mkfifo(const char *pathname, mode_t mode)`
Creates a FIFO special file. Returns 0 on success, -1 on error.
- Regular file operations: `open()`, `read()`, `write()`, `close()`.

How It Works

1. One process creates the FIFO (e.g., `mkfifo("myfifo", 0666)`).
2. The writer opens the FIFO for writing (`open("myfifo", O_WRONLY)`).
The reader opens for reading (`open("myfifo", O_RDONLY)`).
(By default, `open()` blocks until the other end is also opened.)
3. Data is written/read just like a pipe.
4. After communication, both processes close the FIFO. The FIFO file can be removed with `unlink()`.

Example: Writer and Reader

writer.c

```
c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>
#include <string.h>

int main() {
    const char *fifo = "/tmp/myfifo";
    mkfifo(fifo, 0666);           // Create FIFO

    int fd = open(fifo, O_WRONLY);
    char *msg = "Hello via FIFO!";
    write(fd, msg, strlen(msg) + 1);
    close(fd);

    unlink(fifo);                // Remove FIFO
    return 0;
}
```

reader.c

```
c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/stat.h>
#include <unistd.h>

int main() {
    const char *fifo = "/tmp/myfifo";
    char buffer[100];

    int fd = open(fifo, O_RDONLY);
    read(fd, buffer, sizeof(buffer));
    printf("Reader received: %s\n", buffer);
    close(fd);

    return 0;
}
```

Compile & run (in separate terminals):

```
bash
gcc writer.c -o writer
gcc reader.c -o reader
./writer &    # or run in background; will block until reader opens
./reader
```

(If writer runs before reader, it blocks on `open()` until reader opens the FIFO for reading.)

Output (reader):

text

Reader received: Hello via FIFO!

Blocking Behavior

- By default, `open()` for reading blocks until a writer opens, and vice versa. This can be avoided using `O_NONBLOCK`.
 - `read()` blocks if no data is available; `write()` blocks if the pipe is full.
-

4. Shared Memory

Shared memory allows multiple processes to access the same region of memory. It is the fastest form of IPC because data is not copied between kernel and user space – processes simply read/write the shared region. However, it requires explicit synchronization (e.g., using semaphores) to prevent race conditions.

Linux provides two main APIs: **POSIX shared memory** (preferred) and **System V shared memory**. We'll focus on POSIX.

4.1 POSIX Shared Memory

Key System Calls

- `int shm_open(const char *name, int oflag, mode_t mode)`
Creates/opens a shared memory object. Returns a file descriptor.
- `int ftruncate(int fd, off_t length)`
Sets the size of the shared memory object.
- `void *mmap(void *addr, size_t length, int prot, int flags, int fd, off_t offset)`
Maps the shared memory object into the process's address space. Returns a pointer to the mapped area.
- `int munmap(void *addr, size_t length)`
Unmaps the shared memory region.
- `int shm_unlink(const char *name)`
Removes the shared memory object name (like unlink for a file).

Steps to Use Shared Memory

1. **Create/open** a shared memory object with `shm_open()` (using `O_CREAT` flag).
2. **Set its size** with `ftruncate()`.

3. **Map** it into the process address space with `mmap()` (using `PROT_READ` | `PROT_WRITE` and `MAP_SHARED`).
4. Use the returned pointer to access shared data.
5. **Unmap** with `munmap()` when done.
6. **Remove** the shared memory object with `shm_unlink()` (usually done by one process).

4.2 Example: Producer-Consumer

We'll create two programs: a producer that writes an integer to shared memory, and a consumer that reads it. To keep the example simple, we use a flag variable to indicate when data is ready – **this is not thread-safe**; in real code you'd use a mutex or semaphore.

producer.c

```
c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>
#include <string.h>

#define SHM_NAME "/myshm"
#define SHM_SIZE 4096

typedef struct {
    int data;
    int ready;    // 1 = data ready, 0 = not ready (poor man's sync)
} shared_data;

int main() {
    int shm_fd = shm_open(SHM_NAME, O_CREAT | O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }

    // Set size
    if (ftruncate(shm_fd, SHM_SIZE) == -1) {
        perror("ftruncate");
        exit(EXIT_FAILURE);
    }

    // Map shared memory
    shared_data *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                           MAP_SHARED, shm_fd, 0);

    if (shm == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    // Produce data
```

```

shm->data = 42;
shm->ready = 1;    // signal consumer
printf("Producer wrote: %d\n", shm->data);

// Unmap (keep object alive for consumer)
munmap(shm, SHM_SIZE);
// Do not unlink yet - consumer still needs it
return 0;
}

```

consumer.c

```

c
#include <stdio.h>
#include <stdlib.h>
#include <fcntl.h>
#include <sys/mman.h>
#include <unistd.h>

#define SHM_NAME "/myshm"
#define SHM_SIZE 4096

typedef struct {
    int data;
    int ready;
} shared_data;

int main() {
    int shm_fd = shm_open(SHM_NAME, O_RDWR, 0666);
    if (shm_fd == -1) {
        perror("shm_open");
        exit(EXIT_FAILURE);
    }

    shared_data *shm = mmap(NULL, SHM_SIZE, PROT_READ | PROT_WRITE,
                             MAP_SHARED, shm_fd, 0);
    if (shm == MAP_FAILED) {
        perror("mmap");
        exit(EXIT_FAILURE);
    }

    // Wait until data is ready (simple spin-wait - not efficient)
    while (!shm->ready)
        ;    // In real code, use proper synchronization (semaphore)

    printf("Consumer read: %d\n", shm->data);

    // Cleanup
    munmap(shm, SHM_SIZE);
    shm_unlink(SHM_NAME);    // Remove shared memory object
    return 0;
}

```

Compile & run:

```
bash
gcc producer.c -o producer -lrt
gcc consumer.c -o consumer -lrt
./producer & # run in background (or in another terminal)
./consumer
```

Output (consumer):

```
text
Consumer read: 42
```

Notes:

- The `-lrt` flag links the real-time library (for `shm_open` and `shm_unlink`).
- The spin-wait loop is a placeholder; real applications should use synchronization objects like POSIX semaphores.
- Shared memory objects live in a virtual filesystem, typically mounted under `/dev/shm/`. You can list them with `ls /dev/shm`.

4.3 System V Shared Memory (Brief)

System V IPC uses different functions:

- `shmget()` to create/obtain a shared memory segment.
- `shmat()` to attach it to the process address space.
- `shmdt()` to detach.
- `shmctl()` for control operations (including removal).

POSIX shared memory is generally simpler and integrates better with file descriptors and `mmap()`. However, legacy code may still use System V.

5. Comparison: Pipes vs. Shared Memory

Feature	Pipes (including FIFOs)	Shared Memory
Data transfer	Byte stream (kernel buffer)	Direct memory access (zero-copy)
Speed	Slower (data copied between kernel and user)	Fastest (no copying)
Synchronization	Automatic – read blocks if pipe empty, write blocks if pipe full	Manual – must use semaphores, mutexes, etc.
Communication	Unidirectional (or bidirectional with two pipes)	Any direction (both processes can read/write)

Process relationship	Unnamed pipes: only related processes. FIFOs: any processes.	Any processes (with proper permissions)
Persistence	Unnamed: exists only while open. FIFO: exists as file until deleted.	Object exists until explicitly removed (<code>shm_unlink</code>)
Complexity	Low	Higher (needs synchronization)

When to use what?

- **Pipes** are ideal for simple, sequential data streams (e.g., shell pipelines) and when you want automatic blocking behaviour.
- **Shared memory** is best for high-throughput data sharing, especially when large amounts of data are exchanged frequently, and you are prepared to implement proper synchronization.